

A Topology-Aware Remeshing Algorithm for Even-Valence Meshes with Applications in Path-Based Fabrication

Joseph Su

hang.su.25@ucl.ac.uk

Abstract

Triangular remeshing is a fundamental process in computational geometry that modifies polygonal meshes through local operations such as edge splitting, collapsing, flipping, and vertex smoothing. While originally developed for improving numerical stability and geometric quality, its role in architectural computation extends to fabrication-aware design, where topology and connectivity are critical due to material and assembly constraints.

This report investigates remeshing as a topology-aware optimization process under fabrication-oriented scenarios. Graph-theoretic constraints, particularly Eulerian traversability, are used to restructure mesh connectivity for applications such as path-based construction, weaving systems, and checkerboard patterns.

Conventional methods typically aim for near-uniform valence (around six in triangular meshes), improving geometric regularity but offering limited topological control. This study extends remeshing toward enforcing even-valence distributions (e.g., 4, 6, 8...), enabling alternative connectivity structures better aligned with fabrication logic. From this perspective, remeshing becomes an active process that directly influences both geometry and downstream design outcomes.

1. Introduction

In architectural design, digital meshes serve not only as geometric representations but also as frameworks for organizing material and fabrication systems. However, conventional triangular meshes often lack the topological constraints required for fabrication-aware applications, where connectivity is more critical than geometric accuracy.

Triangular remeshing addresses mesh quality through local operations, typically targeting near-uniform valence and isotropic triangle distributions (Botsch and Kobbelt, 2004). While effective for simulation, such approaches provide limited control over topology in fabrication contexts.

This report reinterprets remeshing as a topology-driven process by introducing graph-based constraints, particularly even-valence conditions linked to Eulerian traversability. It investigates how enforcing strict even-valence structures affects mesh behavior and evaluates the resulting configurations across path-based fabrication-oriented scenarios.

2. Isotropic Remeshing Algorithm

2.1 Triangular Mesh and Valence

A triangular mesh $M = (V, E, F)$ is a discrete surface representation defined by a vertex set V , edge set $E \subseteq V^2$, and triangular face set $F \subseteq V^3$, where each edge connects two vertices, and each face is a triplet of vertices. In this formulation, the mesh is explicitly treated as an edge-based representation, where connectivity is directly encoded by E , enabling efficient local topological operations such as edge split, collapse, and flip. The valence of a vertex is defined as the number of incident edges (or adjacent faces) connected to it. As a primary topological descriptor, valence governs local connectivity and strongly influences mesh traversal behavior and structural organization.

2.2 Core Geometric Operators

Triangular remeshing is built upon a set of local operators that modify mesh connectivity and geometry, including edge split, edge collapse, edge flip, and vertex smoothing. This operator-based framework underlies classical mesh optimization methods (Hoppe et al., 1993) and isotropic remeshing pipelines (Botsch and Kobbelt, 2004). Some other operators—such as stellar operators—are later introduced within the operator set as a variant of the edge split operation in remeshing frameworks (Zhang and Panozzo). These operators locally optimize mesh topology through a sequence of incremental edits, iteratively updating geometry to improve mesh quality, maintain surface fidelity, and achieve more regular and well-structured triangulations.

2.2.1 Edge Split

Edge split is a local mesh refinement operator that increases geometric resolution by introducing a new vertex on an existing edge and updating the adjacent connectivity to maintain a valid triangulation. Given an edge $e_i = (v_j, v_k)$, the operation inserts a new vertex v_m at the edge midpoint: $v_m = \frac{v_j + v_k}{2}$. The original edge is then replaced by two new edges (v_j, v_m) and (v_m, v_k) . Each incident face $f = (v_j, v_k, v_a)$ sharing e_i is deleted and retriangulated into two faces: (v_j, v_m, v_a) , (v_m, v_k, v_a) .

```

SPLIT( $e_i = (v_j, v_k), V, E, F$ ):
   $v_m \leftarrow (v_j + v_k)/2$ 
   $V \leftarrow V \cup \{v_m\}$ 
   $E \leftarrow E \cup \{(v_j, v_m), (v_m, v_k)\}$ 
  for each face  $f = (v_j, v_k, v_a)$ 
     $\in F$  that contains  $e_i$  do
     $E \leftarrow E \cup \{(v_m, v_a)\}$ 
     $F \leftarrow F \setminus \{f\}$ 
     $F \leftarrow F \cup \{(v_j, v_m, v_a), (v_m, v_k, v_a)\}$ 
  end for
   $E \leftarrow E \setminus \{e_i\}$ 
  return  $m$ 

```

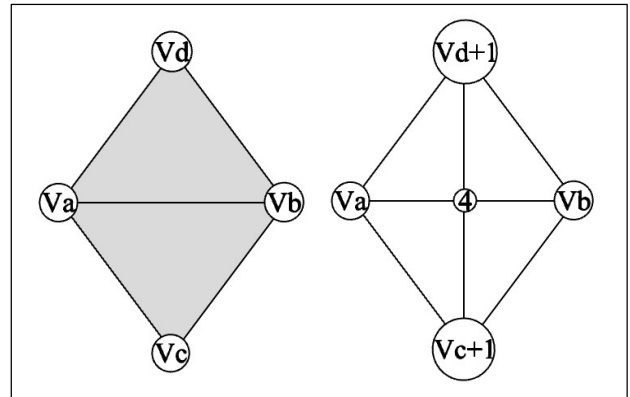


Figure 1: Pseudo-code of the mesh edge split algorithm.

Figure 2: Diagram of the edge split operation.

2.2.2 Edge Collapse

Edge collapse is a local mesh simplification operator that reduces geometric complexity by contracting an edge into a single vertex while updating the surrounding connectivity to preserve a valid triangulation. Given an edge $e_i = (v_j, v_k)$, the operation removes the vertex v_k and merges its local neighborhood into v_j . All edges incident to v_k , except the collapsing edge (v_j, v_k) , are reassigned to v_j , provided that duplicate edges are avoided. The incident faces are then updated accordingly. Faces containing the collapsing edge e_i are removed, while all other faces incident to v_k are retriangulated by replacing v_k with v_j . Any resulting degenerate configurations are implicitly discarded, ensuring that the mesh remains manifold and consistent.

```

COLLAPSE( $e_i = (v_j, v_k), V, E, F$ ):
  for each edge  $(v_k, v_a) \in E, a \neq j$  do
     $E \leftarrow E \setminus \{(v_k, v_a)\}$ 
    if  $(v_j, v_a) \notin E$  then
       $E \leftarrow E \cup \{(v_j, v_a)\}$ 
    end if
  end for
  for each face  $f = (v_k, v_a, v_b) \in F$  do
     $F \leftarrow F \setminus \{f\}$ 
    if  $a \neq j, b \neq j$  then
       $F \leftarrow F \cup \{(v_j, v_a, v_b)\}$ 
    end if
  end for
   $V \leftarrow V \setminus \{v_k\}$ 
  return  $j$ 

```

Figure 3: Pseudo-code of the mesh edge collapse algorithm.

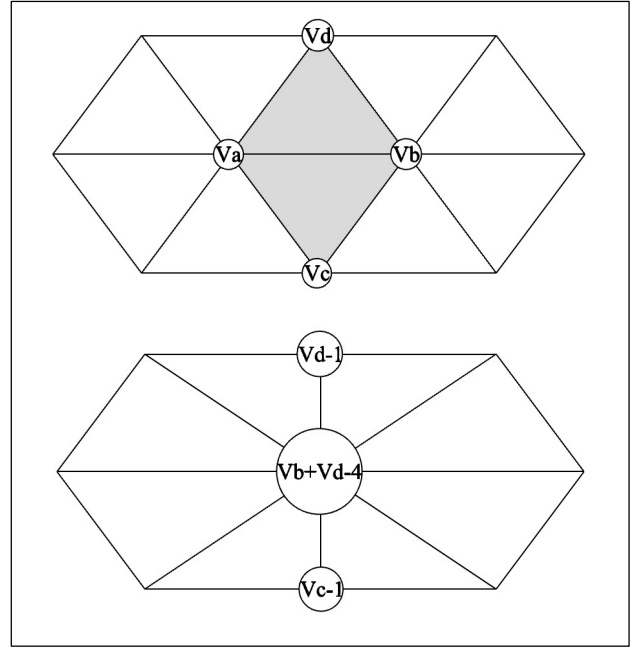


Figure 4: Diagram of the edge collapse operation.e.

2.2.3 Edge Flip

Edge flip is a local connectivity optimization operator that improves mesh quality without modifying vertex positions. It operates on an interior edge by reconfiguring the connectivity of the two adjacent triangles. Given an edge $e_i = (v_j, v_k)$, the set of adjacent faces sharing this edge is first identified. If exactly two faces are incident to e_i , denoted as $f_1 = (v_j, v_k, v_m)$ and $f_2 = (v_k, v_j, v_n)$, the edge is considered flippable. The operation replaces the original edge (v_j, v_k) with the opposite diagonal (v_m, v_n) , provided that this edge does not already exist. The two incident faces are removed and replaced by two new faces (v_m, v_n, v_j) and (v_n, v_m, v_k) , thereby preserving a valid triangulation.

```

FLIP( $e_i = (v_j, v_k), V, E, F$ ):
  AdjFaces  $\leftarrow \{f \in F \mid e_i \subset f\}$ 
  if  $|AdjFaces| = 2$  then
    let  $f_1 \leftarrow (v_j, v_k, v_m) \in AdjFaces$ 
    let  $f_2 \leftarrow (v_k, v_j, v_n) \in AdjFaces$ 
    if  $(v_m, v_n) \notin E$  then
       $E \leftarrow E \cup \{(v_m, v_n)\}$ 
    end if
     $F \leftarrow F \cup \{(v_m, v_n, v_j), (v_n, v_m, v_k)\}$ 
     $E \leftarrow E \setminus \{e_i\}$ 
     $F \leftarrow F \setminus AdjFaces$ 
  end if

```

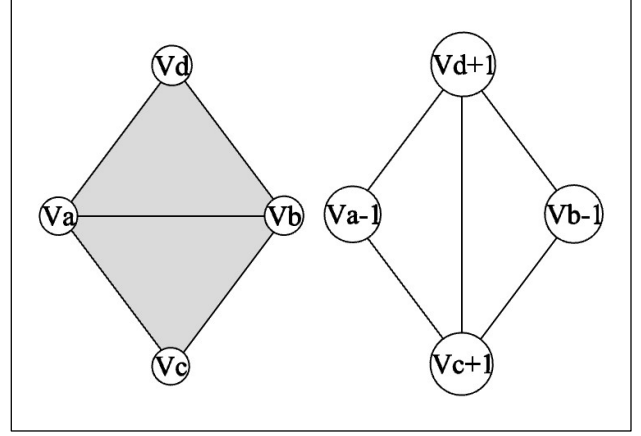


Figure 5: Pseudo-code of the mesh edge flip algorithm.
Figure 6: Diagram of the edge flip operation.

2.2.4 Tangential Relaxation and Laplacian Smoothing

Laplacian smoothing is employed as a local vertex update operator to improve geometric regularity while preserving surface characteristics. For a vertex v_i , the Laplacian smoothing operator computes a displacement toward the barycenter of its one-ring neighborhood $N(i)$, defined as: $L(v_i) = \frac{1}{|N(i)|} \sum_{v_j \in N(i)} (v_j - v_i)$.

```

LAPLACIAN_SMOOTH( $v_i, V, E$ ):
  let  $N(i) \leftarrow \{v_j \mid (v_i, v_j) \in E\}$ 
   $c \leftarrow \mathbf{0}$ 
  for each vertex  $v \in N(i)$  do
     $c \leftarrow c + v$ 
  end for
   $c \leftarrow c / |N(i)|$ 
   $L \leftarrow c - v_i$ 
  return  $L$ 

```

```

TANGENTIAL_RELAXATION( $\lambda, L$ ):
  compute vertex normal  $n_i$ 
   $T \leftarrow I - n_i n_i^T$ 
   $\Delta v \leftarrow \lambda T L$ 
  return  $\Delta v$ 

```

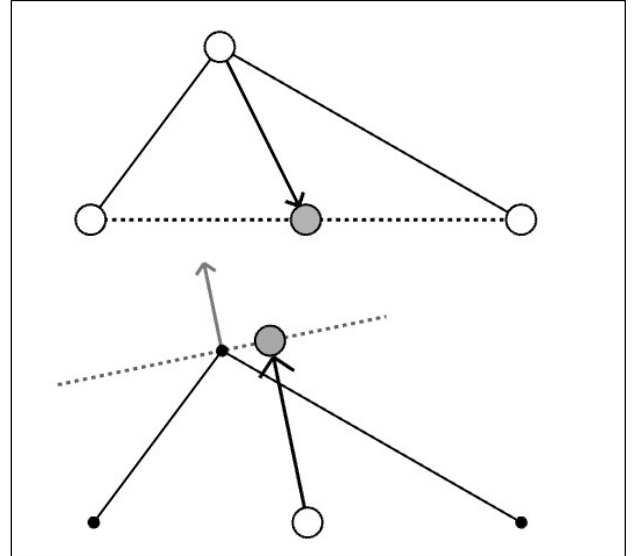


Figure 7: Pseudo-code of the Laplacian smooth algorithm.
Figure 8: Pseudo-code of the tangential relaxation algorithm.
Figure 9: Diagrams of the Laplacian smooth and tangential relaxation.

While this promotes local regularization, it may introduce shrinkage if applied directly. To mitigate this, tangential relaxation operator projects the Laplacian displacement onto the local tangent plane using the projection operator: $T(v_i) = I - n_i n_i^T$, where n_i is the vertex normal. The updated vertex position is then given by: $v_i' = v_i + \lambda T(v_i) L(v_i)$, or equivalently in combined form: $v_i' = v_i + \lambda (I - n_i n_i^T) \left(\frac{1}{|N(i)|} \sum_{v_j \in N(i)} v_j - v_i \right)$. To preserve geometric constraints, this update is applied only to unconstrained vertices, such that $v_i' = v_i$ if $v_i \in V_a$ where V_a denote the sets of anchored vertices.

2.3 Isometric Remeshing Objectives

Remeshing aims to improve both the geometric quality and structural regularity of a triangular mesh. A primary objective is the equalization of edge lengths toward a target value L_0 , typically enforced within a tolerance range $L_{\min} = \epsilon_1 L_0$ and $L_{\max} = \epsilon_2 L_0$. This promotes uniform sampling and reduces poorly shaped elements. Closely related to this is the formation of near-equilateral triangles, which improves numerical stability and minimizes geometric distortion in surface representation. Another key objective is the regulation of $\text{VALENCE}(v_i)$, defined as the number of incident edges at a vertex, which in isotropic triangular meshes tends to converge toward $\text{VALENCE}(v_i) \approx 6$, a consequence of global topological constraints described by Euler's formula (Euler characteristic), $V - E + F = \chi$, which implies that triangular meshes exhibit an average vertex valence of approximately 6. In adaptive remeshing strategies, the target edge length becomes spatially varying, expressed as $L_0 = L(x) = f(\kappa(x))$, where $\kappa(x)$ denotes local curvature and $f(\cdot)$ defines a mapping from curvature to sampling density. Feature preservation is also critical, maintaining sharp geometric details during remeshing. The constraint set C represents sharp edges, boundary curves, and feature lines. Feature vertices $V_f \subseteq V$ lie on the constraint set C , while anchor vertices $V_a \subseteq V$ remain fixed during optimization.

2.4 Isometric Remeshing Implementation

The implementation of remeshing is typically formulated as an iterative procedure started by the target edge length L_0 , from which an acceptable edge length range $(L_{\min}, L_{\max}) = L_0(\epsilon_1, \epsilon_2)$ is defined. In the remeshing method proposed by Botsch and Kobbelt (2004), specific parameter choice of $(\epsilon_1, \epsilon_2) = (\frac{4}{5}, \frac{4}{3})$ are reported to yield stable and high-quality isotropic remeshing results. Within each iteration, local operators are applied sequentially based on this criterion.

Edges $E = \{e_0, e_1, \dots, e_n\}$ are adaptively updated based on length constraints. For each edge $e_i = (v_j, v_k) \in E$ with length $L(e_i)$, the remeshing procedure enforces: $L(e_i) > L_{\max} \Rightarrow \text{SPLIT}$, $L(e_i) < L_{\min} \Rightarrow \text{COLLAPSE}$. For feature preservation, the collapse point is repositioned conditionally based on the relationship between the endpoints and the constraint vertices. This can be expressed as:

$$\text{COLLAPSE_POINT}(e_i, V_f, V_a) = \begin{cases} \text{MIDPOINT}(v_j, v_k), & (v_j, v_k) \cap (V_f \cup V_a) = \emptyset \\ v_j, & (v_j, v_k) \cap (V_f \cup V_a) = \{v_k\} \\ v_k, & (v_j, v_k) \cap (V_f \cup V_a) = \{v_j\} \\ \emptyset, & |(v_j, v_k) \cap (V_f \cup V_a)| = 2 \end{cases},$$

where V_f and V_a denote the sets of feature and anchored vertices respectively.

FLIP is subsequently applied to improve local connectivity by reconfiguring the diagonal of adjacent triangle pairs, thereby reducing local valence irregularities. Consider an interior edge $e_i = (v_j, v_k)$ shared by two triangular faces (v_j, v_k, v_m) and (v_k, v_j, v_n) . The flip operation evaluates the valence quality of the four involved vertices through a local energy measure: $E_{\text{current}} = \sum_{v \in \{v_j, v_k, v_m, v_n\}} (\text{VALENCE}(v) - 6)^2$. After performing a candidate flip that replaces edge (v_j, v_k) with (v_m, v_n) , the updated configuration yields: $E_{\text{flipped}} = \sum_{v \in \{v_j, v_k, v_m, v_n\}} (\text{VALENCE}'(v) - 6)^2$. FLIP is accepted if it reduces the local valence error, i.e.: $E_{\text{flipped}} < E_{\text{current}}$.

This is followed by vertex repositioning using TANGENTIAL_RELAXATION, which is formulated as a

LAPLACIAN_SMOOTH step projected onto the local tangent plane, where vertex updates are computed as the projected Laplacian displacement and applied only to unconstrained vertices. Specifically, TANGENTIAL_RELAXATION is only performed if $v_i \notin V_a$, ensuring that only free vertices undergo relaxation.

Finally, vertex projection is performed to maintain geometric fidelity. For feature vertices $v_i \in V_f$, each vertex is projected onto its corresponding feature curve or constraint set, while non-feature vertices $v_i \notin V_f \cup V_a$ are projected onto the target mesh surface to preserve overall shape consistency.

This iterative process continues until the mesh satisfies the desired geometric and topological conditions, resulting in a balanced and well-structured triangulation.

Algorithm 1: Isotropic Remeshing Loop
input:
 $M = (V, E, F)$: **triangular mesh** to be modified

 $V_a \subseteq V$: **set** of anchor vertices

 $V_f \subseteq V$: **set** of feature vertices

 C : **set** of feature

 $solverIterations \in \mathbb{N}^+$
 $S \leftarrow \text{Surface}(V, F)$
 $\epsilon_1 \leftarrow 4/5$
 $\epsilon_2 \leftarrow 3/4$
 $n \leftarrow 0$
while $n < solverIterations$ **do**
for each edge $e_i = (v_j, v_k) \in E$ **do**
 $L_0 \leftarrow L(e_i)$
 $L \leftarrow \text{LENGTH}(e_i)$
 $L_{min} \leftarrow \epsilon_1 L_0$
 $L_{max} \leftarrow \epsilon_2 L_0$
if $L > L_{max}$ **then**
 $\text{SPLIT}(e_i, V, E, F)$
else if $L < L_{min}$ **then**
 $v' \leftarrow \text{COLLAPSE_POINT}(e_i, V_f, V_a)$
if $v' \neq \emptyset$ **then**
 $j \leftarrow \text{COLLAPSE}(e_i, V, E, F)$
 $v_j \leftarrow v'$
end if
end if
end for
for each edge $e_i = (v_j, v_k) \in E$ **do**
 $AdjFaces \leftarrow \{f \in F \mid e_i \subset f\}$
if $|AdjFaces| = 2$ **then**
 $\text{let } f_1 \leftarrow (v_j, v_k, v_m) \in AdjFaces$
 $\text{let } f_2 \leftarrow (v_k, v_j, v_n) \in AdjFaces$
 $E_{current} \leftarrow 0$
 $E_{flipped} \leftarrow 0$
for each $v \in \{v_j, v_k, v_m, v_n\}$ **do**
 $E_{current} \leftarrow E_{current} + (\text{VALENCE}(v) - 6)^2$
 $E_{flipped} \leftarrow E_{flipped} + (\text{VALENCE}'(v) - 6)^2$
end for
if $E_{flipped} < E_{current}$ **then**
 $\text{FLIP}(e_i, V, E, F)$
end if
end if
end for
 $V' \leftarrow V$
for each vertex $v_i \in V \setminus V_a$ **do**
 $L \leftarrow \text{LAPLACIAN_SMOOTH}(v_i, V, E)$
 $v'_i \leftarrow v_i + \text{TANGENTIAL_RELAXATION}(\lambda, L)$
 $V' \leftarrow V' \setminus \{v_i\}$
 $V' \leftarrow V' \cup \{v'_i\}$
end for
for each vertex $v'_i \in V'$ **do**
if $v'_i \in V_f$ **then**
 $v'_i \leftarrow \Pi_C(v'_i)$
else
 $v'_i \leftarrow \Pi_S(v'_i)$
end if
end for
 $n \leftarrow n + 1$
end while
Figure 10: Pseudo-code of the isotropic remeshing algorithm.

3. Example Application of Topology-Aware Remeshing in Architectural Computation: Kagome Weaving Structures

3.1 Project Overview

The use of topology-aware mesh processing has already been explored within computational architectural fabrication workflows. In this context, Ayres, Martin, and Zwierzycki (2018) present a computational framework for generating and fabricating weaving structures, specifically kagome (triaxial) weave patterns, directly from mesh topology. The method enables the transformation of abstract surface geometry into physically constructible interlaced lattice structures composed of straight strips, thereby linking discrete geometric processing with fabrication constraints in a unified design pipeline (Ayres et al., 2018).

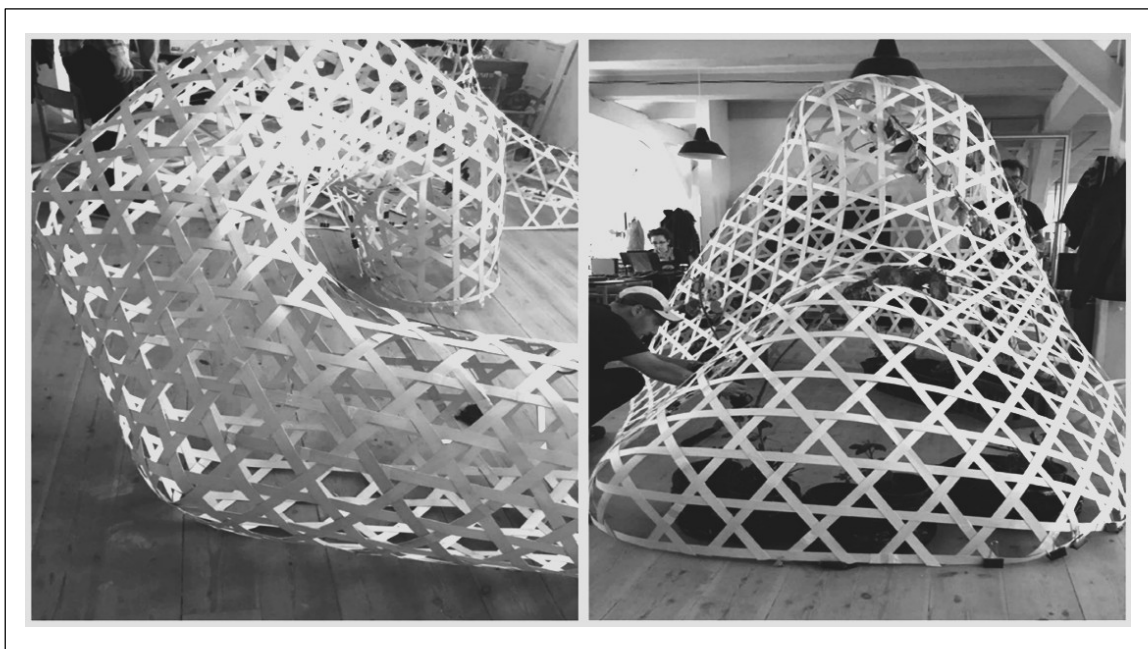


Figure 11: Kagome weaving structure. *Image reproduced from [6].*

The proposed workflow begins with a coarse triangulated mesh representing the target architectural surface. This mesh is not intended as a final geometric representation but rather as a topological proxy encoding global form. Connectivity is therefore prioritized over geometric precision, and the mesh functions as a control structure for subsequent operations (Ayres et al., 2018).

In the next stage, the mesh is analyzed through vertex valence, where the number of incident edges at each vertex is used as an indicator of local curvature behavior. Vertices with valence approximately equal to six correspond to locally flat regions, while lower and higher valence values are associated with positive and negative curvature respectively. In this way, curvature is encoded implicitly through topology rather than explicit geometric formulation (Ayres et al., 2018).

To prepare the mesh for weaving generation, local topological operations such as edge split and edge collapse are applied. These operations adjust vertex valence, redistribute connectivity, and regularize irregular regions of the mesh. Edge splitting increases resolution and enables finer curvature control, while edge collapse reduces complexity and removes redundant structure. Importantly, this stage relies on discrete topological editing rather than geometric smoothing,

reinforcing the topology-driven nature of the approach (Ayres et al., 2018).

A physics-based relaxation process is then applied to approximate material behavior. In this simulation, strips are modeled as elastic elements governed by constraints that preserve length, manage intersections, and minimize bending deformation. While this improves physical plausibility, the relaxation process may also introduce deviations from the intended geometric configuration due to internal stress redistribution (Ayres et al., 2018).

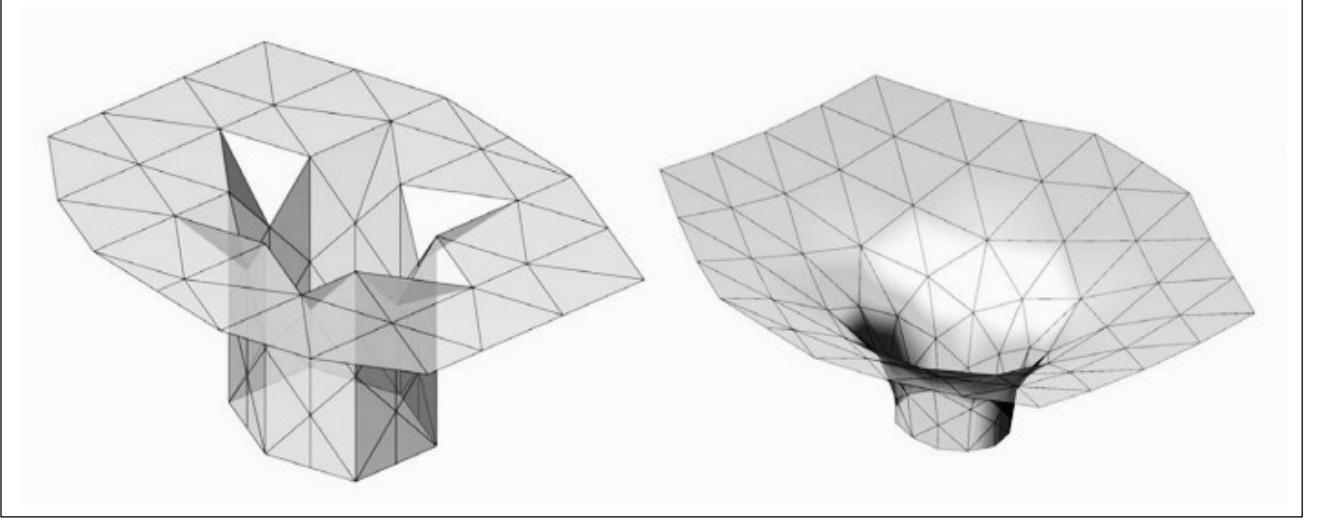


Figure 12: Diagram of physical relaxation. *Image reproduced from [6].*

Once a consistent topology is established, a dual graph representation is constructed by mapping mesh faces to graph nodes and adjacency relations to edges. This dual structure serves as the basis for generating the kagome weave pattern, in which a triangular interlaced network emerges naturally from the underlying connectivity structure (Ayres et al., 2018).

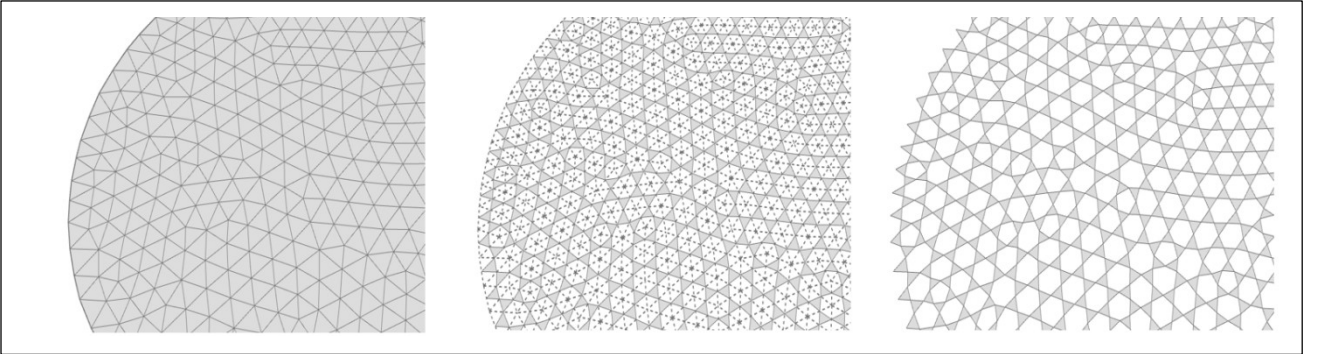


Figure 13: Diagram that shows the generation of triangular interlaced network.

Weaving paths are subsequently extracted by tracing continuous edge sequences through the dual graph, producing strip-based elements that define the physical weave. Each strip is constrained to remain geometrically straight, which introduces challenges in maintaining continuity across curved regions of the surface. This stage translates abstract graph connectivity into manufacturable strip layouts and crossover sequences (Ayres et al., 2018).

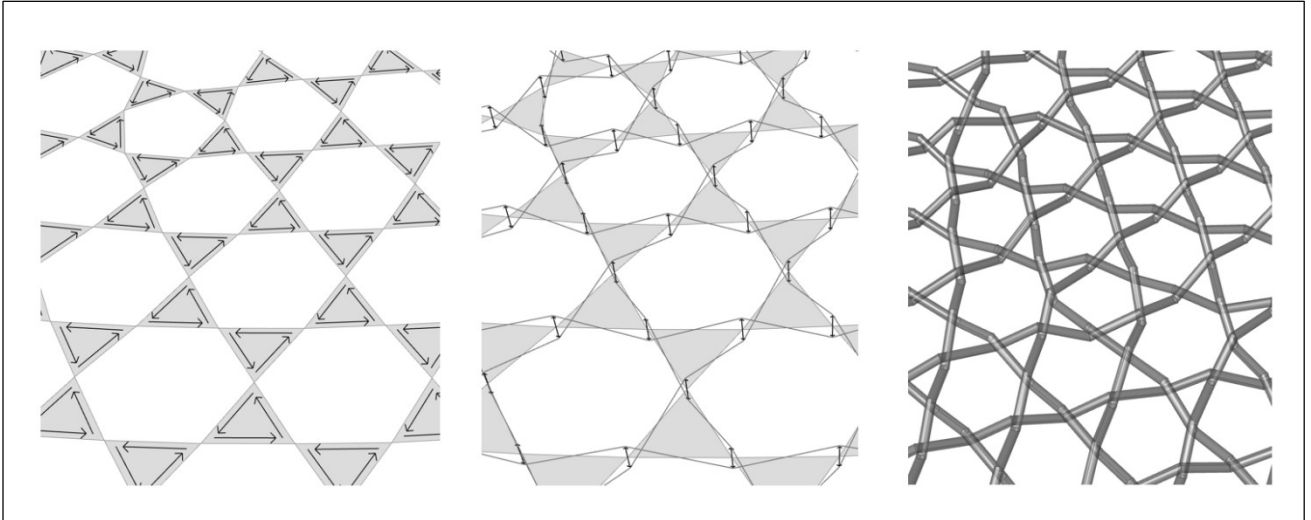


Figure 14: Diagram that shows the generation of manufacturable strips and crossover sequences.

Finally, the system outputs fabrication data including strip lengths, weaving order, and assembly instructions. The resulting structure is a constructible interlaced lattice derived entirely from topology-driven operations rather than explicit surface modeling (Ayres et al., 2018).

3.2 Critical Evaluation

The work demonstrates several key strengths. First, it reconceptualizes kagome weave structures as computationally derivable systems directly governed by mesh topology, thereby integrating architectural form, discrete geometry, and fabrication logic within a unified framework (Ayres et al., 2018). Second, it establishes a topology-aware mesh processing pipeline in which vertex valence and dual graph structure directly determine the resulting weave configuration. Third, the use of local split and collapse operations provides a mechanism for controlled modification of curvature distribution, enabling a form of shape control through topological editing rather than continuous geometric optimization.

However, several limitations remain. The paper does not provide rigorous structural validation of the resulting lattice systems, such as mechanical performance analysis or load-bearing evaluation (Ayres et al., 2018). In addition, the kagome formulation is restricted to a two-strip crossover configuration, resulting in a triangular–polygonal interlaced structure rather than a fully generalized triangular weave. As a consequence, the system cannot generate a complete and fully flexible triangulated pattern, which may reduce its potential structural efficiency and physical performance. This restriction also limits the overall generality and extensibility of the weaving framework. The assumption of globally straight strips introduces torsional effects at crossings when mapped onto curved geometries, potentially leading to physical inconsistency in fabrication. Furthermore, while mesh editing operations are introduced, the workflow still requires non-trivial manual intervention, particularly in remeshing and topology adjustment, limiting full automation. Finally, the relaxation process is not fully integrated into the topology generation stage, which can result in geometric drift between the designed and physically relaxed configurations.

4. Proposed Topology-Aware Remeshing Algorithm: Even-Valence Remeshing

The previous framework for kagome weaving relies significantly on manual mesh editing and guided topological adjustments, particularly during remeshing and valence correction. This dependence limits its automation potential and reduces scalability in complex design scenarios. To address this limitation, the following chapter focuses on topology-aware remeshing algorithms as a means of enabling more spontaneous and computationally driven mesh adaptation across multiple architectural fabrication scenarios.

4.1 Graph-Theoretic Foundation

From a graph-theoretic perspective, enforcing even vertex degree is fundamental, as it guarantees the existence of an Eulerian circuit on a connected graph, enabling a single continuous traversal of all edges without repetition. This unconditional traversability under a closed walk provides a globally consistent ordering independent of traversal initialization. It is particularly important in architectural fabrication, where continuous toolpath generation, material deposition, and robotic execution require such deterministic traversal sequences to ensure structural continuity and manufacturing feasibility.

A triangular mesh $M = (V, E, F)$ can be interpreted as an undirected graph, where vertices V correspond to nodes and edges E define adjacency. The valence of a vertex $v_i \in V$, denoted $\deg(v_i)$, is defined as the number of incident edges. A fundamental property of graphs is given by the handshaking lemma: $\sum_{v_i \in V} \deg(v_i) = 2|E|$. Since the sum of all vertex degrees is even, and even-degree vertices contribute only even values, the sum of degrees over odd-valence vertices must also be even. As each contributes an odd value, their count must be even: $|V_{odd}| \equiv 0 \pmod{2}$, where $V_{odd} = \{v_i \in V \mid \deg(v_i) \equiv 1 \pmod{2}\}$. Therefore, by pairing odd vertices and applying local topological operations (e.g., edge split, collapse, and flip) along their connecting shortest paths, the mesh can be modified to achieve globally even valence.

4.2 Second-Order Vertex Traversal via Opposite-Edge Expansion

To eliminate the uneven valence vertices in triangular meshes by remeshing, we introduce a set of operators that define a second-order traversal in the mesh dual graph based on opposite-edge face expansion, specifically designed to support the edge-based operations in subsequent remeshing stages.

4.2.1 Opposite Edge and Vertex

Given a triangular face $f = (v_0, v_1, v_2)$, with $v_i \in V$, we introduce the notions of opposite edge and opposite vertex based on the combinatorial structure of a triangle. For a vertex $v \in f$, the opposite edge is defined as the edge in f does not contain v , which can be expressed as: $e_{opp}(f, v) = f \setminus \{v\}$. Conversely, for an edge $e \subset f$, the opposite vertex is defined as the vertex in f does not contain e , which can be expressed as: $v_{opp}(f, e) = f \setminus e$.

4.2.2 Face Transition Operator

We further define the face transition operator f_{opp} to describe the adjacency across a shared edge. For a face $f \in F$ and an edge $e \subset f$, the operator returns the adjacent face: $f_{opp}(e, f) = f'$, such that $e = f \cap f'$, $f' \in F$ (i.e., f' is the unique face incident to e on the opposite side of f). This operator enables traversal in mesh dual graph via edge-based face expansion.

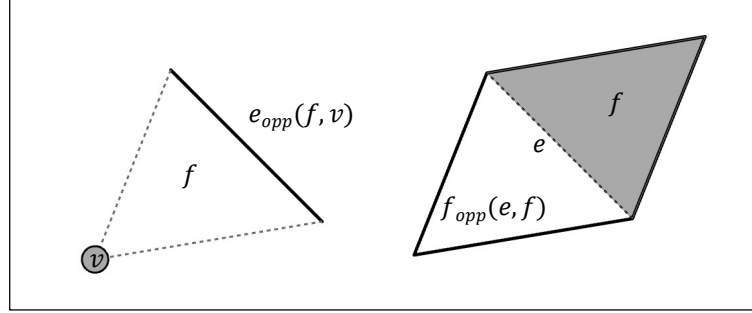


Figure 15: Diagrams of the opposite edge and face transition operator.

4.2.3 Second-Order Face Set via Opposite-Edges

The incident face set (i.e., 1-ring face neighborhood) of a vertex $v \in V$ is given by $F_1(v) = \{f \in F \mid v \in f\}$. Using the opposite-edge operator e_{opp} and the face transition operator f_{opp} , we extend the local context to a second-order face set defined as: $F_2(v) = \{f_{opp}(e_{opp}(f, v), f) \mid f \in F_1(v)\}$. This operator performs a dual-graph expansion across edges opposite to v , yielding a structured second-order face neighborhood.

4.2.4 Second-Order Vertex Set via Opposite-Edges

With the opposite-edge operator e_{opp} , opposite vertex v_{opp} and face transition operator f_{opp} , we further construct a second-order vertex set by extracting vertices opposite to the traversal edges in the expanded face set, which is expressed as: $V_2^{opp}(v) = \{v_{opp}(f_{opp}(e, f), e) \mid f \in F_1(v), e = e_{opp}(f, v)\}$. This operator maps each incident face of v through opposite-edge expansion to an adjacent face, and selects the vertex opposite to the corresponding edge, yielding a structured second-order vertex neighborhood.

4.3 Even-Valence Remeshing Implementation

4.2.4 BFS Pairing on Second-Order Vertex Traversal

To address irregular connectivity induced by odd-valence vertices, we perform a constrained BFS over V_{odd} , guided by second-order opposite-edge face expansion. For each unpaired $v \in V_{odd}$, the traversal follows the operator chain: $w = v_{opp}(f_{opp}(e_{opp}(f, u), f), e_{opp}(f, u))$, which is the discrete form of $V_2^{opp}(v)$ operator.

A predecessor structure ($vertexParent[\cdot]$, $edgeParent[\cdot]$) is maintained to encode the BFS tree for path reconstruction between paired vertices. When a target vertex $w \in V_{odd}$ the connecting edge path is recovered via backtracking along the predecessor links. The resulting edges are accumulated into E_{path} , and their incident faces are collected into F_{path} .

4.2.5 Global Valence Correction

The UPDATE_VALENCE procedure performs adaptive remeshing along an edge path E_{path} to regularize vertex valence. The boundary vertex set V_b is first defined as boundary vertices. For each edge $e_i \in E_{path}$, the adjacent face set F_{adj} is retrieved and the incident vertices v_j, v_k, v_m, v_n are identified. Their valences V_1, V_2, V_3, V_4 are evaluated with boundary correction via $V = 6$ for $v \in V_b$. A local condition evaluating the current valences determines the operation: if satisfied, collapse e_i ; otherwise a valid collapse point v' is computed and the edge is collapsed if feasible. This alternation of SPLIT and COLLAPSE enforces valence regularization while preserving mesh consistency.

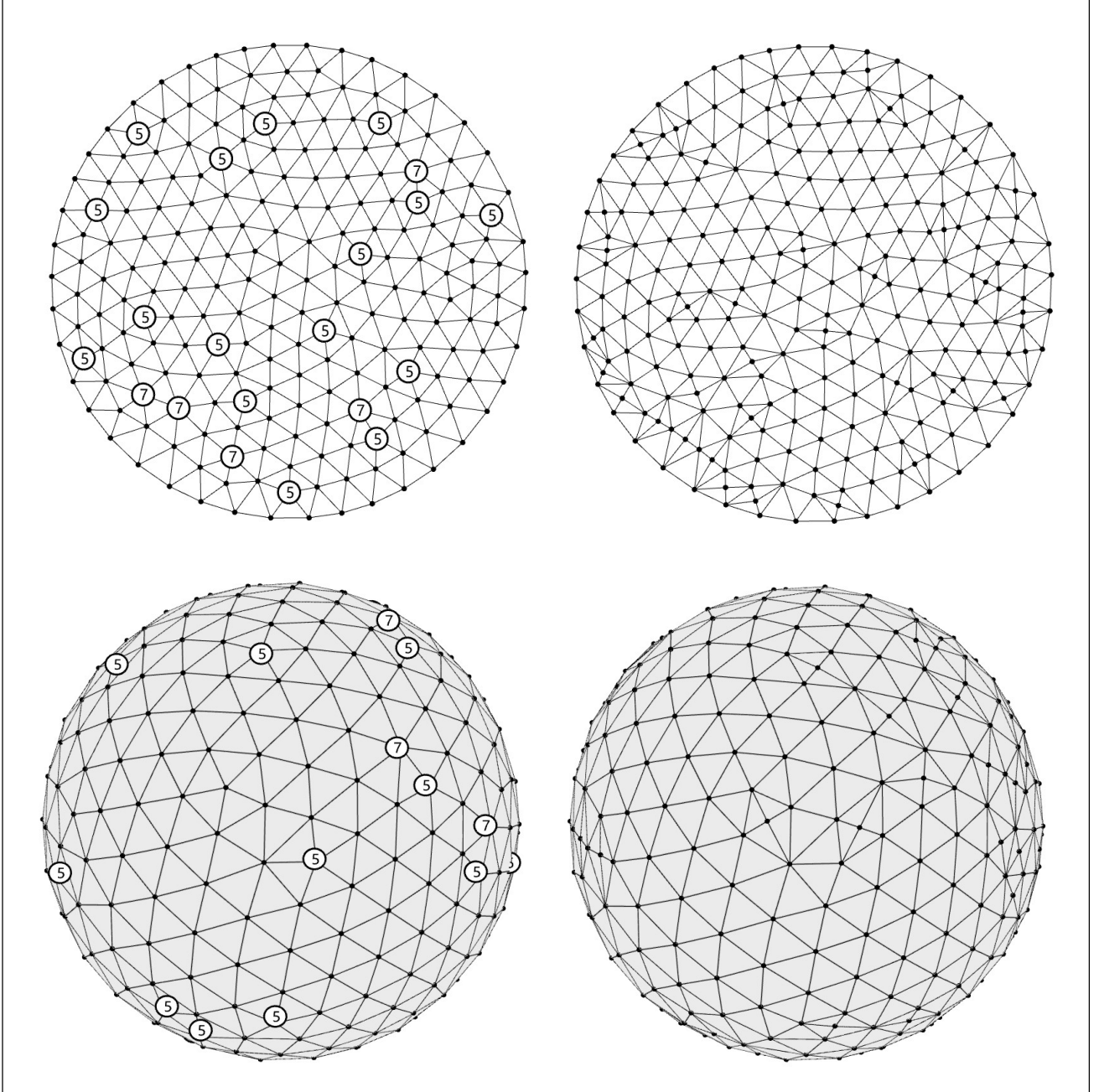


Figure 16: Results of even valence remeshing algorithm. The upper row shows the comparison of an open mesh before and after remeshing, while the lower row shows the comparison of a closed mesh before and after remeshing.

```

ODD_VERTEX_PAIR( $V, E, F$ ):
 $V_{odd} \leftarrow \{v \in V \mid \text{VALENCE}(v) \equiv 1 \pmod{2}\}$ 
 $F_{path} = \emptyset$ 
 $E_{path} = \emptyset$ 
 $V_{pair} \leftarrow \emptyset$ 
for each  $v \in V_{odd}$  do
    if  $v$  is not paired then
         $Q \leftarrow \{v\}$ 
         $V_{vis} \leftarrow \emptyset$ 
         $vertexParent[v] \leftarrow \perp$ 
         $edgeParent[v] \leftarrow \perp$ 
        while  $Q \neq \emptyset$  do
             $u \leftarrow \text{dequeue}(Q)$ 
            for each  $f \in F_1(u)$  do
                 $e \leftarrow e_{opp}(f, u)$ 
                 $f' \leftarrow f_{opp}(e, f)$ 
                 $w \leftarrow v_{opp}(f', e)$ 
                if  $f \notin F_{path} \wedge f' \notin F_{path} \wedge f' \neq \emptyset$  then
                    if  $w \notin V_{vis} \cup V_{pair}$  then
                         $V_{vis} \leftarrow V_{vis} \cup \{w\}$ 
                         $vertexParent[w] \leftarrow u$ 
                         $edgeParent[w] \leftarrow e$ 
                         $\text{enqueue}(Q, w)$ 
                        if  $w \in V_{odd}$  then
                             $V_{pair} \leftarrow V_{pair} \cup \{v, w\}$ 
                             $x \leftarrow w$ 
                            while  $x \neq v$  do
                                 $e' \leftarrow edgeParent[x]$ 
                                 $E_{path} \leftarrow E_{path} \cup \{e'\}$ 
                                 $F_{path} \leftarrow F_{path} \cup \{f \in F \mid e' \subset f\}$ 
                                 $x \leftarrow vertexParent[x]$ 
                            end while
                            break
                        end if
                    end if
                end if
            end for
        end while
    end if
end for
return  $E_{path}$ 

```

Figure 17: Pseudo code of odd valence vertices pairing algorithm.

```

UPDATE_VALENCE( $E_{path}, V, E, F, V_f, V_a$ ):
 $V_b \leftarrow \{v \in V \mid \exists e \in E, v \in e, |\{f \in F \mid e \subset f\}| = 1\}$ 
for each edge  $e_i \in E_{path}$  do
     $AdjFaces \leftarrow \{f \in F \mid e_i \subset f\}$ 
    if  $|AdjFaces| = 2$  then
        let  $f_1 \leftarrow (v_j, v_k, v_m) \in AdjFaces$ 
        let  $f_2 \leftarrow (v_k, v_j, v_n) \in AdjFaces$ 

         $V1 \leftarrow (v_m \in V_b) ? 6 : \text{VALENCE}(v_m)$ 
         $V2 \leftarrow (v_n \in V_b) ? 6 : \text{VALENCE}(v_n)$ 
         $V3 \leftarrow (v_j \in V_b) ? 6 : \text{VALENCE}(v_j)$ 
         $V4 \leftarrow (v_k \in V_b) ? 6 : \text{VALENCE}(v_k)$ 

         $oppVTooDense \leftarrow$ 
             $(V1 \geq 6 \vee V2 \geq 6) \wedge (V1 + V2 > 12)$ 
         $collapseSafe \leftarrow$ 
             $V3 + V4 < 12$ 
         $collapseCondition \leftarrow$ 
             $oppVTooDense \wedge collapseSafe$ 

        if  $collapseCondition$  then
             $v' \leftarrow \text{COLLAPSE\_POINT}(e_i, V_f, V_a)$ 
            if  $v' \neq \emptyset$  then
                 $\text{COLLAPSE}(e_i, V, E, F)$ 
                 $v_j \leftarrow v'$ 
            end if
        else
             $\text{SPLIT}(e_i, V, E, F)$ 
        end if
    end if
end for

```

Figure 18: Pseudo code of odd valence correction algorithm.

5.EVR Evaluation in Design Scenarios

To evaluate the effectiveness of the proposed topology-aware remeshing algorithm, this chapter examines its applications across representative architectural design scenarios in graph-theoretic formulations.

The first scenario considers weaving generation based on edge traversability in the primal graph, corresponding to Eulerian-type traversal where all edges are covered continuously. Two approaches can be used to generate a triangular-pattern weaving structure. One approach identifies local ring edge pairs as path-in and path-out constraints, followed by simulated annealing or other optimization methods to approximate a globally optimal weaving configuration. The alternative approach constructs a global Eulerian circuit and traverses all segments to produce a zero-error weaving configuration incrementally. The proposed algorithm is then evaluated on both closed spherical meshes and open free-form meshes generated via a growth-based algorithm. In both cases, the method produces uniformly distributed and stable triangular weaving configurations.

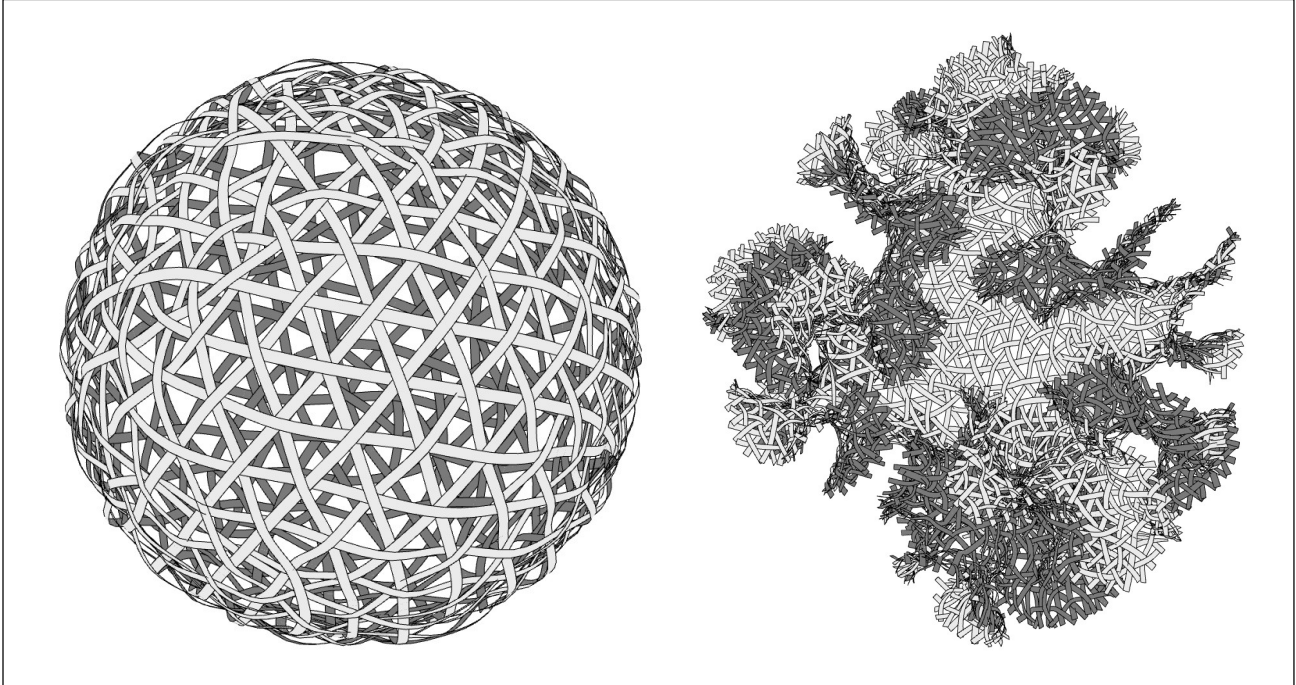


Figure 19: Results of triangular weaving generation based on EVR. The left is produced on a closed spherical mesh. The right is produced on a free-form mesh generated via a growth-based algorithm.

The second scenario examines checkerboard generation via face traversability in the dual graph. This is formulated as a graph coloring problem on G^* , where faces are nodes and adjacency is defined by shared edges. A valid checkerboard requires G^* to be bipartite, i.e., a mapping $c: F \rightarrow \{0,1\}$ such that adjacent faces have different labels. This is equivalent to the absence of odd cycles in G^* , i.e., bipartiteness. Vertex valence provides local regularity in the primal mesh, producing an even cyclic ordering of incident edges and faces around each vertex.

Both scenarios—checkerboard pattern generation and weaving pattern generation—can be applied to architectural façade design and other fabrication-oriented design contexts, offering solutions that are both geometrically expressive and physically constructible. This demonstrates the general applicability of the proposed framework across different architectural design scenarios.

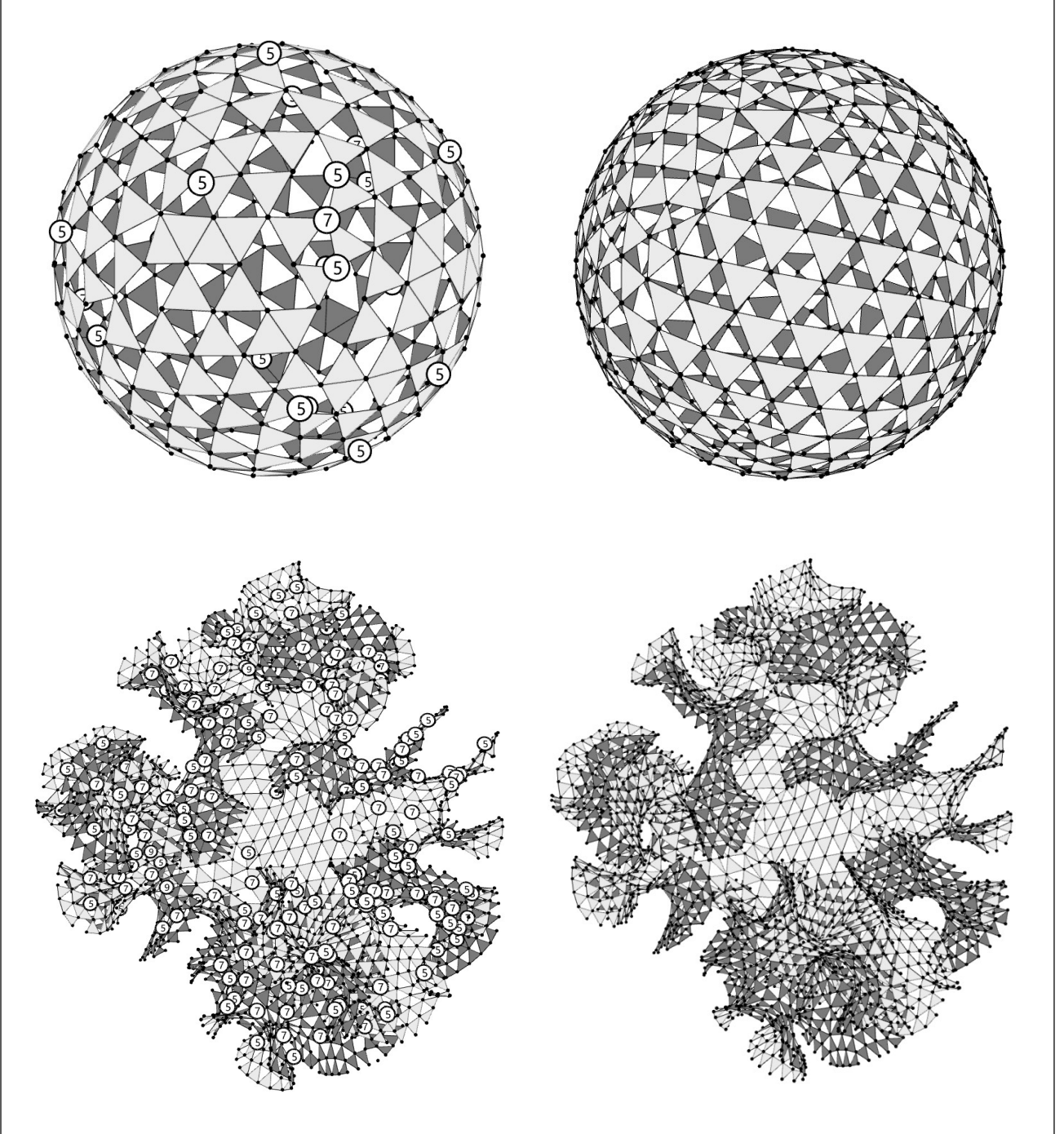


Figure 20: Results of triangular checkerboard coloring generation. The upper row shows the comparison on a closed spherical mesh before and after EVR. The lower row shows the comparison on a free-form mesh generated via a growth-based algorithm before and after EVR.

6. Discussion of EVR

The proposed even-valence correction operates as a global, one-shot operator applied to an already constructed mesh. This is in contrast to conventional remeshing methods, which typically follow local and incremental optimization strategies, analogous to greedy or hill-climbing procedures. Due to this fundamental difference in formulation, the even-valence correction cannot be embedded within the iterative loop of standard remeshing algorithms. If applied concurrently, the valence correction objective may conflict with local geometric optimization targets, leading to

instability or divergence in the remeshing process. As a result, the operator must be applied as a post-processing step after conventional remeshing has converged.

A second limitation arises from the second-order vertex traversal mechanism employed in the algorithm. Since traversal progresses by skipping one vertex at each step, vertices separated by odd-length distances cannot be consistently paired. This structural constraint can leave residual odd configurations in the mesh, preventing complete enforcement of even-valence conditions in certain topological cases.

7. Conclusion

The proposed even-valence remeshing framework demonstrates strong potential as a foundational tool for topology-aware architectural design. By introducing a global correction mechanism for mesh valence, the method supports a range of geometric and fabrication-driven design contexts, in which graph-theoretic structure plays a central role. Eulerian and Hamiltonian formulations, where fabrication processes can be interpreted respectively as edge-covering and vertex-covering traversals of mesh graphs. Eulerian paths correspond to continuous edge traversal, supporting material continuity in weaving systems and encoding global ordering constraints for single-path fabrication strategies such as 3D printing.

More importantly, the framework reduces reliance on manual mesh manipulation, enabling more automated and systematic design workflows. This shift has the potential to free designers from low-level geometric correction tasks, allowing greater focus on higher-level form finding, structural logic, and design exploration. In this way, the approach supports both creative exploration and constructible outcomes, bridging the gap between computational generation and fabrication feasibility.

Reference:

- [1] M. Botsch and L. Kobbelt, “A remeshing approach to multiresolution modeling,” in Proc. Eurographics/ACM SIGGRAPH Symp. Geometry Processing, 2004, pp. 185–192.
- [2] H. Hoppe, T. DeRose, T. Duchamp, J. McDonald, and W. Stuetzle, “Mesh optimization,” in Proc. 20th Annu. Conf. Comput. Graph. Interactive Techniques, 1993, pp. 19–26.
- [3] V. Surazhsky and C. Gotsman, “Explicit surface remeshing,” in Proc. Eurographics/ACM SIGGRAPH Symp. Geometry Processing, 2003, pp. 20–30.
- [4] Y. Zhang and D. Panozzo, “Direct triangle mesh remeshing using stellar operators,” in geometry processing literature (commonly cited in SIGGRAPH/Eurographics context), n.d.
- [5] J. Vorsatz, C. Rössl, and H.-P. Seidel, “Dynamic remeshing and applications,” in Proc. 8th ACM Symp. Solid Modeling and Applications, G. Elber and V. Shapiro, Eds. New York, NY, USA: ACM, 2003, pp. 167–175.
- [6] P. Ayres, A. G. Martin, and M. Zwierzycki, “Beyond the basket case: A principled approach to the modelling of kagome weave patterns for the fabrication of interlaced lattice structures using straight strips,” 2018.